

Practical Week 11

Aim

To develop a multithreaded counter application using POSIX threads (pthread_create() and pthread_join()) and demonstrate a race condition when multiple threads update a shared variable concurrently.

Program:

```
GNU nano 8.7.1 race_condition.c *
pthread_t threads[NUM_THREADS];

for (int i = 0; i < NUM_THREADS; i++)
{
    pthread_create(&threads[i], NULL, increment_counter, NULL);
}

for (int i = 0; i < NUM_THREADS; i++)
{
    pthread_join(threads[i], NULL);
}

printf("Expected counter value: %d\n",
      NUM_THREADS * ITERATIONS);

printf("Actual counter value: %d\n", counter);

return 0;
}
```

Output:

```
navaneeth@Navaneeth: ~/practical_week11
navaneeth@Navaneeth:~/practical_week11$ mkdir -p ~/practical_week11
navaneeth@Navaneeth:~/practical_week11$ cd ~/practical_week11
navaneeth@Navaneeth:~/practical_week11$ nano race_condition.c
navaneeth@Navaneeth:~/practical_week11$ gcc race_condition.c -o race_condition -pthread
navaneeth@Navaneeth:~/practical_week11$ ./race_condition
Expected counter value: 400000
Actual counter value: 137594
navaneeth@Navaneeth:~/practical_week11$ ./race_condition
Expected counter value: 400000
Actual counter value: 116392
navaneeth@Navaneeth:~/practical_week11$ ./race_condition
./race_condition
./race_condition
./race_condition
Expected counter value: 400000
Actual counter value: 133143
Expected counter value: 400000
Actual counter value: 153680
Expected counter value: 400000
Actual counter value: 109003
Expected counter value: 400000
Actual counter value: 168061
navaneeth@Navaneeth:~/practical_week11$ |
```

Observation

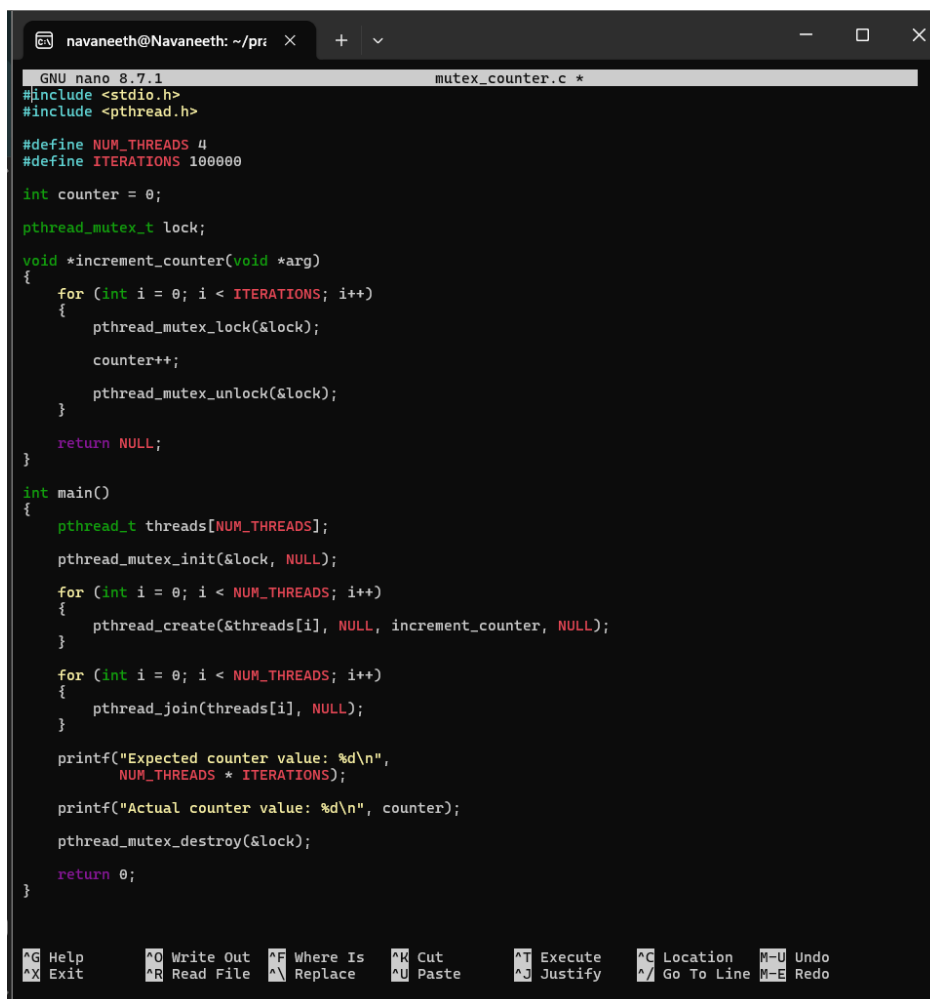
Multiple POSIX threads were created using `pthread_create()`. The threads simultaneously modified the shared counter variable. Because no synchronization mechanism was used, a race condition could occur and the final counter value may differ from the expected value.

Result

A multithreaded counter application was successfully implemented using POSIX threads, and the race condition caused by concurrent access to a shared variable was demonstrated.

Aim

To synchronize access to a shared variable using a POSIX mutex and eliminate the race condition.



```
GNU nano 8.7.1 mutex_counter.c *
#include <stdio.h>
#include <pthread.h>

#define NUM_THREADS 4
#define ITERATIONS 100000

int counter = 0;
pthread_mutex_t lock;

void *increment_counter(void *arg)
{
    for (int i = 0; i < ITERATIONS; i++)
    {
        pthread_mutex_lock(&lock);
        counter++;
        pthread_mutex_unlock(&lock);
    }
    return NULL;
}

int main()
{
    pthread_t threads[NUM_THREADS];
    pthread_mutex_init(&lock, NULL);

    for (int i = 0; i < NUM_THREADS; i++)
    {
        pthread_create(&threads[i], NULL, increment_counter, NULL);
    }

    for (int i = 0; i < NUM_THREADS; i++)
    {
        pthread_join(threads[i], NULL);
    }

    printf("Expected counter value: %d\n",
        NUM_THREADS * ITERATIONS);

    printf("Actual counter value: %d\n", counter);

    pthread_mutex_destroy(&lock);
    return 0;
}
```

Help Write Out Where Is Cut Execute Location M-U Undo
Exit Read File Replace Paste Justify Go To Line M-E Redo

```

navaneeth@Navaneeth:~/practical_week11$ nano mutex_counter.c

navaneeth@Navaneeth:~/practical_week11$ gcc mutex_counter.c -o mutex_counter -pthread
navaneeth@Navaneeth:~/practical_week11$ ./mutex_counter
Expected counter value: 400000
Actual counter value: 400000
navaneeth@Navaneeth:~/practical_week11$ ./mutex_counter
./mutex_counter
./mutex_counter
./mutex_counter
Expected counter value: 400000
Actual counter value: 400000
Expected counter value: 400000
Actual counter value: 400000
Expected counter value: 400000
Actual counter value: 400000
Expected counter value: 400000
Actual counter value: 400000
navaneeth@Navaneeth:~/practical_week11$ ./race_condition
Expected counter value: 400000
Actual counter value: 135779
navaneeth@Navaneeth:~/practical_week11$ ./mutex_counter
Expected counter value: 400000
Actual counter value: 400000
navaneeth@Navaneeth:~/practical_week11$ |

```

Observation

Without synchronization, multiple threads can access the shared counter simultaneously, causing a race condition. With the mutex, access to the shared counter is protected, and the final value consistently reaches the expected value.

Result

The race condition was successfully eliminated using POSIX mutex synchronization, and the synchronized program produced the expected counter value.